

APM1110_FA5_Libres_Ben Alexander

Github Repository: <https://github.com/benalexander/libres>

Item 1: Discrete Random Variable

To find the constant k , we set the sum of all PMF values to 1 using the property $\sum P(X = x) = 1$.

```
x <- 1:4
weights <- 2 * x + 1

# (a) Solve for k and calculate PMF
k <- 1 / sum(weights)
pmf <- k * weights

# Sanity check: ensure valid PMF
stopifnot(all.equal(sum(pmf), 1))

# (b) Expected Value E[X] & Variance Var(X)
E_X <- sum(x * pmf)
E_X2 <- sum((x^2) * pmf)
Var_X <- E_X2 - (E_X)^2

# (c) P(X >= 3)
p_ge_3 <- sum(pmf[x >= 3])

# Output results
cat("a) Constant k =", k, "(or 1/24)\n")
```

```
## a) Constant k = 0.04166667 (or 1/24)
```

```
cat("b) E[X] =", round(E_X, 4), "| Var(X) =", round(Var_X, 4), "\n")
```

```
## b) E[X] = 2.9167 | Var(X) = 1.0764
```

```
cat("c) P(X >= 3) =", round(p_ge_3, 4), "\n")
```

```
## c) P(X >= 3) = 0.6667
```

Item 2: Binomial Distribution

Analyzing a sample of $n = 20$ requests with malicious probability $p = 0.08$.

```

n <- 20
p <- 0.08

# (a) Exactly 2 malicious requests
p_eq_2 <- dbinom(2, size = n, prob = p)

# (b) At least 3 malicious requests
p_ge_3 <- pbinom(2, size = n, prob = p, lower.tail = FALSE)

# (c) Mean and Standard Deviation
mean_x <- n * p
sd_x <- sqrt(n * p * (1 - p))

# Output results
cat("a) P(X = 2) =", round(p_eq_2, 4), "\n")

```

```
## a) P(X = 2) = 0.2711
```

```
cat("b) P(X >= 3) =", round(p_ge_3, 4), "\n")
```

```
## b) P(X >= 3) = 0.2121
```

```
cat("c) Mean =", round(mean_x, 4), "| Standard Deviation =", round(sd_x, 4), "\n")
```

```
## c) Mean = 1.6 | Standard Deviation = 1.2133
```

Item 3: Poisson Distribution

Modeling web traffic with an arrival rate of $\lambda = 4.5$ hits per minute.

```

# (a) Exactly 6 hits in 1 minute
p_eq_6 <- dpois(6, lambda = 4.5)

# (b) At most 2 hits in 1 minute
p_le_2 <- ppois(2, lambda = 4.5)

# (c) More than 10 hits in 2 minutes (scaled lambda = 9.0)
p_gt_10 <- ppois(10, lambda = 9.0, lower.tail = FALSE)

# Output results
cat("a) P(X = 6) =", round(p_eq_6, 4), "\n")

```

```
## a) P(X = 6) = 0.1281
```

```
cat("b) P(X <= 2) =", round(p_le_2, 4), "\n")
```

```
## b) P(X <= 2) = 0.1736
```

```
cat("c) P(Y > 10) =", round(p_gt_10, 4), "\n")
```

```
## c) P(Y > 10) = 0.294
```

Item 4: Normal Distribution

Query processing time follows a normal distribution $N(\mu = 120, \sigma = 15)$.

```
mu <- 120
sigma <- 15

# (a) P(X > 135)
p_gt_135 <- pnorm(135, mean = mu, sd = sigma, lower.tail = FALSE)

# (b) P(100 <= X <= 130)
p_between <- pnorm(130, mean = mu, sd = sigma) - pnorm(100, mean = mu, sd = sigma)

# (c) 95th Percentile
p95 <- qnorm(0.95, mean = mu, sd = sigma)

# Output results
cat("a) P(X > 135) =", round(p_gt_135, 4), "\n")
```

```
## a) P(X > 135) = 0.1587
```

```
cat("b) P(100 <= X <= 130) =", round(p_between, 4), "\n")
```

```
## b) P(100 <= X <= 130) = 0.6563
```

```
cat("c) 95th Percentile =", round(p95, 4), "ms\n")
```

```
## c) 95th Percentile = 144.6728 ms
```

```
# Density visualization
curve(dnorm(x, mean = mu, sd = sigma), from = 70, to = 170,
      main = "Database Query Processing Time",
      xlab = "Time (ms)", ylab = "Density", col = "steelblue", lwd = 2)
abline(v = 135, col = "red", lty = 2, lwd = 1.5)
```

Database Query Processing Time

